How LLMs Work: Pretraining

Jim Fawcett · conversation with Claude · October 18, 2026

FIRST PROMPT

> I want to explore how LLM AI models work. I will ask a series of questions about that. Please correct my concepts as needed. Think carefully before replying. First Question:
>
> During learning a vector of learning data is injected into the model and that is used to optimize parameters. Then that process is repeated with a new vector of learning data. That continues for very many passes until the parameter optimization has reached something approximating steady state. Is this correct? If not clarify.

FINAL PROMPT

> Give a detailed summary of all of the information about pretraining you've given me.

REPLY

Pretraining: Summary

1. What pretraining is

Pretraining is the first and by far the largest phase of building an LLM. The model is a large parameterized function, and training repeatedly adjusts its parameters, typically billions of numbers that are mostly weight matrices, so that it gets better at one task: **predicting the next token of text**. The outcome is a strong next-token predictor, not yet an assistant.

2. The data

- **Source.** A huge corpus, often trillions of tokens. It mixes:
  - filtered web crawl (the bulk)
  - code
  - books, academic papers, reference works, and math
  - multilingual text
  - licensed and increasingly synthetic data
- Frontier recipes are mostly unpublished; open models document the general pattern.
- **Curation.** Text is deduplicated and filtered for quality, spam, safety, and privacy.
- **Weighted mixture.** Sources are sampled in chosen proportions. High-quality sources may be repeated, and low-quality ones downsampled. Late in training the mix often shifts toward the highest-quality data (annealing or mid-training). Data selection is one of the most important levers on model quality.

- **The corpus defines the knowledge.** The model knows roughly what the corpus contained, in proportion to how often and how consistently it appeared. Gaps, biases (language, viewpoint, era), and the knowledge cutoff all follow from the data.
- **Formatting into sequences.** Documents are tokenized into integer IDs and separated by end-of-document tokens. They are packed into fixed-length windows matching the context length (e.g., 8k tokens), shuffled, and grouped into **batches**. A batch is a 2-D array of integers (batch size × sequence length), often totaling millions of tokens.

3. The forward pass ("feeding data through the model")

Evaluating the model function on a batch, using the current parameters. Data flows one direction, through each layer once, with no loops. Intermediate values (**activations**) are temporary; nothing from the data is stored in the model during this step.

1. **Embedding.** Each token ID selects a row of a learned embedding matrix. This is where integers become vectors (e.g., 4,096 numbers per token).
2. **A stack of N layers** (dozens to 100+), each with its own parameters. Each layer updates every position's vector:
   - **Attention.** Each position computes a query, a key, and a value via learned matrices. It compares its query to the keys of *earlier* positions only, because of the **causal mask**. It then takes a relevance-weighted sum of their values. This is the only mechanism that moves information between positions.
   - **MLP (feed-forward).** Each position's vector passes independently through a small two-layer network. Much of the model's stored “knowledge” appears to live here.
   - **Residual stream.** Each sub-step's output is *added* to the running vector, with normalization keeping the scale stable. Layers read from and write refinements into this stream.
3. **Unembedding.** Final vectors are multiplied by an output matrix, giving a score (**logit**) for every vocabulary entry (100k+).
4. **Softmax.** Logits become a probability distribution over the next token at each position.

Nearly all of this is large matrix multiplication, which is why GPUs and TPUs are used.

4. The objective and loss

- **Next-token prediction at every position at once.** Because of the causal mask, one forward pass predicts token 2 from token 1, token 3 from tokens 1–2, and so on. A 4,000-token sequence yields about 4,000 training examples in parallel.
- **Cross-entropy loss:** the average over positions of **−log P(actual next token | all preceding tokens)**.
  - The log heavily punishes *confident* mistakes.
  - Because softmax probabilities sum to 1, raising the correct token's probability implicitly lowers all alternatives. The model is only ever told which token was right.
- **What the signal is not.** It never says “be helpful” or “be true.” Grammar, facts, and reasoning patterns emerge only because they improve next-token prediction.

5. The update step

- **Backpropagation** runs the computation graph in reverse to get the **gradient**: how much the loss would change for a small change in each parameter, including the embedding table.
- **The optimizer** (usually **AdamW**) moves every parameter a small step *opposite* its gradient, downhill on the loss. AdamW keeps running averages of past gradients and scales each parameter's step individually.
- **Stochastic gradient descent.** Each batch gives only a noisy estimate of the true downhill direction, so each step is a small nudge. The optimization happens through the accumulation of hundreds of thousands of such steps.
- **Distribution.** Each batch is split across thousands of accelerators, and their gradients are averaged before the update.

6. When it stops

- **Roughly one pass over the data.** Most data is seen about once, with selected high-quality sources repeated. Training is not looping a fixed dataset to convergence.
- **The budget, not a steady state, ends it.** Loss is typically still falling slowly at the end, following fairly predictable **scaling laws**. Training stops when the planned compute budget is spent.
- **Apparent settling.** The **learning rate schedule** warms up, then decays toward a small value. This damping makes the parameters settle, but that is not a true optimum.
- **What success means.** Success is measured by loss on **held-out data** the model never trained on (generalization), not by parameters ceasing to change.

7. Compact statement

> *Pretraining repeatedly evaluates the model function on batches of token sequences drawn from a large, curated, weighted corpus. At every position it measures how much probability the model gave to the actual next token given its prefix (cross-entropy loss). Backpropagation computes that loss's gradient with respect to every parameter, and the optimizer moves all parameters a small step to lower it. This continues for hundreds of thousands of steps over trillions of tokens seen about once, ending when the compute budget is used, with the learning rate decayed so the parameters settle.*

8. Connections beyond pretraining

- **Post-training** uses much smaller, purpose-built data with different objectives:
  - supervised fine-tuning on example conversations
  - reinforcement learning from preference judgments
  - reinforcement learning on tasks with checkable outcomes
- Pretraining shapes what the model **knows**; post-training shapes mostly how it **behaves**.

- **Inference** uses the same forward pass with frozen parameters. The model computes the next-token distribution, samples a token, appends it, and repeats. There is no backpropagation and no update.